

KnowledgeHub — Explanation Guide

A plain-language walkthrough of the whole project System Design • Semester 4

This guide explains, in simple words but with real technical depth, **what we built, why each file exists, which architecture we follow, and how the system answers the standard system-design questions**. It is meant to be read top-to-bottom before a presentation or viva.

Companion documents: [KnowledgeHub_Documentation.pdf](#) (the formal report) and [KnowledgeHub_Code_Walkthrough.pdf](#) (a **line-by-line explanation of every file** — what each import is and where it comes from). This guide is the "explain it to me like I'm presenting it" version.

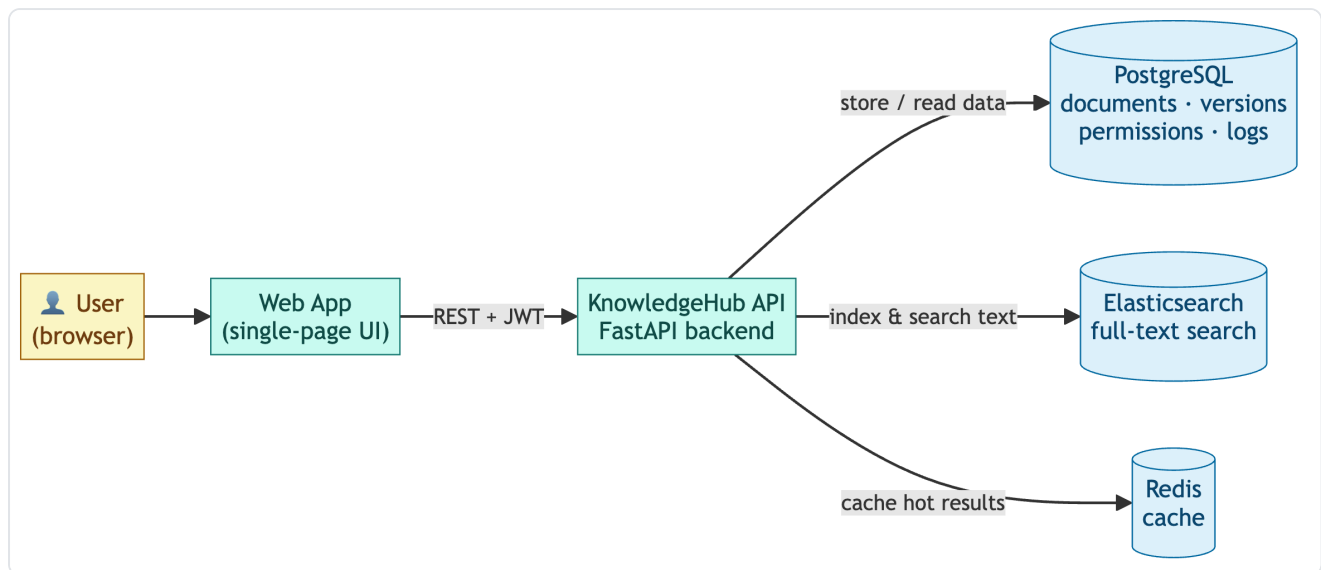
Table of Contents

1. What is KnowledgeHub? (the one-paragraph version)
 2. The big picture (simple architecture diagram)
 3. Which architecture are we following?
 4. The technology choices, in plain words
 5. File-by-file: why each file exists
 6. How a request actually flows (two end-to-end stories)
 7. The six descriptive questions (Q1–Q6), answered simply
 8. System-design interview questions, answered *for this project*
 9. Quick revision cheat-sheet
-

1. What is KnowledgeHub? (the one-paragraph version)

KnowledgeHub is a **company knowledge base** — think Confluence or Notion. Employees write documents (guides, policies, FAQs), and other employees **search** for them and **collaborate** (comment, see history, control who can read what). The hard part is doing this **fast and safely when there are millions of documents and thousands of users at once**. Our project builds a working version of this: a web app, a backend API, a database, a search engine, and a cache — wired together so it stays fast, secure, and reliable.

2. The big picture



Read it left to right:

1. **User** opens the **Web App** in a browser.
2. The Web App calls the **KnowledgeHub API** (our FastAPI backend) over HTTP, sending a **JWT token** to prove who they are.
3. The API uses **three specialised data stores**, each doing the one job it is best at: - **PostgreSQL** = the *source of truth*. It stores documents, every old version, who can access what, comments, and logs. - **Elasticsearch** = the *search engine*. It keeps a special "inverted index" so keyword search is near-instant. - **Redis** = the *cache*. It remembers recent answers so repeated requests are returned in microseconds.

That's the entire system in one sentence: **the API coordinates three stores — a database for truth, a search engine for speed, and a cache for repeated reads.**

3. Which architecture are we following?

Short answer: a Modular Monolith that is microservices-ready.

Let's unpack that, because it's a common viva question.

Monolith vs. Microservices — the two extremes

	Monolith	Microservices
What it is	One application, one deployable unit	Many small apps, each deployed separately
Pros	Simple to build, test, deploy; fast calls inside one process	Independent scaling, independent teams, fault isolation
Cons	Everything scales together; one big codebase	Network calls everywhere; complex ops (service discovery, tracing)

What we actually built: a Modular Monolith. - It is **one FastAPI application** (one process, one deployable) — that makes it a *monolith*. - But internally it is split into **clear modules**: `auth`, `documents`, `search`, `collaboration`, plus

service/data layers. Each module has a single responsibility and talks to the others through clean function/router boundaries — that makes it *modular*.

Why this is the right choice for this project: - For a project of this size, microservices would add huge complexity (separate deployments, network calls, service discovery) for **no real benefit**. - Because the modules are already cleanly separated, we can **"graft" any module into its own microservice later** with minimal rewrite. For example, the Search Service or Indexing Pipeline could become a separate service when search traffic justifies independent scaling. Our architecture diagrams (Figure 2 in the report) deliberately draw these as separate boxes for exactly this reason.

The other key architectural pattern: Polyglot Persistence. We do **not** force one database to do everything. We use: - a **relational DB (PostgreSQL)** for structured, related data that needs ACID guarantees, and - a **search engine (Elasticsearch)** for full-text search, - a **key-value store (Redis)** for caching.

This "use the right tool for each job" approach is called **polyglot persistence**, and it's how real systems like Confluence are built (relational DB + Lucene/Elasticsearch).

One more pattern worth naming: CQRS-lite (Command Query Responsibility Segregation). Writes go to PostgreSQL (the command side / source of truth). Reads for search go to Elasticsearch (the query side / optimised read model). The two are kept in sync by the indexing step. We don't implement full CQRS, but the *idea* — separate the write model from the read model — is exactly what we do.

💡 Why a *stateless* API matters for KnowledgeHub

Our API is **stateless** — it keeps no memory of users between requests. Each request carries its own proof of identity (a **JWT token**), so the server verifies who you are without storing any session. Why this is a big deal for *this* project:

- **Scales to millions of concurrent users (Q6).** Because every API copy is interchangeable, we simply **run more copies** behind the load balancer when traffic grows — horizontal scaling. A stateful design couldn't do this cleanly.
- **Fault tolerant (Q6).** If one API copy crashes, **nobody is logged out and no work is lost** — the user's token still works and the load balancer routes their next request to a healthy copy. (A stateful server would lose every session in its memory.)
- **Zero-downtime deploys.** We can replace API copies one at a time during an update; users never notice, because any copy can serve any request.
- **Clean separation = safe restarts.** All real state lives in **PostgreSQL, Elasticsearch and Redis**, not in the API. That's literally why our `kh-api` container can be rebuilt or scaled freely without losing a single document — the API is disposable, the data stores are durable.

One-liner: *"A stateless API lets me run many interchangeable copies behind a load balancer, so KnowledgeHub scales to millions of users, survives a server crash without logging anyone out, and redeploys with zero downtime — because all the real state lives in Postgres, Elasticsearch, and Redis."*

4. The technology choices, in plain words

Tech	What it is	Why we picked it
FastAPI	Python web framework	Fast, modern, and auto-generates API docs (the Swagger page). Easy to read.
PostgreSQL	Relational SQL database	Rock-solid, supports transactions (ACID), perfect for documents + permissions + history that relate to each other.
Elasticsearch	Search engine	Built specifically for full-text search; ranks results by relevance (BM25) in milliseconds.
Redis	In-memory cache	Microsecond reads; takes load off the database for repeated queries.
SQLAlchemy	Python ORM	Lets us define tables as Python classes instead of raw SQL.
JWT + bcrypt	Auth tokens + password hashing	Stateless login (no server-side sessions) + passwords are never stored in plain text.
Docker Compose	Runs all services together	One command (<code>docker compose up</code>) starts the whole stack identically on any machine.
Vanilla HTML/JS	The web UI	No build step, easy to read, served directly by the API.

A nice touch — graceful fallbacks. If Elasticsearch, Redis, or Postgres aren't running, the app automatically falls back to a **pure-Python search index**, an **in-memory cache**, and **SQLite**. So it runs anywhere, even with zero infrastructure — great for grading. `GET /health` tells you which backends are live.

5. File-by-file: why each file exists

Here is the whole project, file by file, in the order it makes sense to understand it.

The application core (`app/`)

File	Why it exists (plain language)
<code>app/config.py</code>	One place for all settings (database URL, secret key, Redis URL...). Reads from environment variables, with safe defaults so it runs out-of-the-box.
<code>app/database.py</code>	Sets up the connection to PostgreSQL (or SQLite) and hands out database "sessions" to the rest of the app.
<code>app/models.py</code>	The database design. Defines every table as a Python class: <code>User</code> , <code>Space</code> , <code>Document</code> , <code>DocumentVersion</code> , <code>Permission</code> , <code>Comment</code> , <code>CollaborationLog</code> . This is the heart of Q4.
<code>app/schemas.py</code>	Defines the shape of API requests and responses (what JSON goes in and out). Validates input automatically.
<code>app/auth.py</code>	Everything about security : hashing passwords (bcrypt), creating/checking JWT tokens, and deciding if a user is allowed to do something (RBAC + per-document permissions).
<code>app/cache.py</code>	Talks to Redis to store and fetch cached results. If Redis is missing, it quietly uses an in-memory dictionary instead.
<code>app/search.py</code>	Talks to Elasticsearch — creates the index, adds/updates/deletes documents, and runs search queries. If Elasticsearch is missing, it uses our own Python index instead.
<code>app/main.py</code>	The entry point . Starts the FastAPI app, connects all the routes, mounts the web UI, and exposes <code>/health</code> .

The brain of search (`app/services/`)

File	Why it exists
<code>app/services/inverted_index.py</code>	The from-scratch search algorithm (Q5). A pure-Python inverted index with TF-IDF scoring and cosine similarity. This is the file to show when asked "how does search actually work?" It also doubles as the offline fallback engine.
<code>app/services/document_service.py</code>	The rules that span multiple stores : when a document changes, it writes a new version, updates the search index, clears the cache, and logs the action — all in one place so the route handlers stay simple.

The API endpoints (`app/routers/`)

File	Why it exists
<code>app/routers/auth.py</code>	The <code>/auth/register</code> , <code>/auth/login</code> , <code>/auth/me</code> endpoints.
<code>app/routers/documents.py</code>	Create, read, update, delete documents; view version history; share a document with another user.
<code>app/routers/search.py</code>	The <code>/search</code> endpoint — checks the cache, filters by what the user can see, queries Elasticsearch.
<code>app/routers/collaboration.py</code>	Comments and the per-document activity feed.

The frontend (`app/static/`)

File	Why it exists
<code>app/static/index.html</code>	The web app — a single page with login, document creation, search, document view, version history, and comments. It's the "Web App" box in the diagram, served by the API at <code>/app</code> .

Infrastructure & ops

File	Why it exists
<code>docker-compose.yml</code>	Defines and links the four containers (API, Postgres, Elasticsearch, Redis) so the whole stack starts with one command.
<code>Dockerfile</code>	Recipe to build the API into a container image.
<code>requirements.txt</code>	The exact Python libraries (and versions) the project needs.
<code>.env.example</code>	A template of the environment variables; copy to <code>.env</code> and fill in.

Demo, tests & docs

File	Why it exists
<code>smoke_test.py</code>	An automated end-to-end test that runs the whole flow and checks every feature works (no external services needed).
<code>notebooks/demo.ipynb</code>	A Jupyter notebook that explains and runs the search algorithm, then drives the full API, with outputs saved for screenshots.
<code>scripts/demo_transcript.py</code>	Runs a clean end-to-end demo against the live stack (used to generate the demo screenshot).
<code>scripts/build_pdf.py</code>	Turns the Markdown docs into the final PDFs.
<code>docs/diagrams/*</code>	The architecture, ER, and sequence diagrams (Mermaid sources + rendered PNGs).
<code>docs/screenshots/*</code>	Real captures of the running web app, Swagger UI, health check, and demo.
<code>README.md</code>	How to set up and run the project.
<code>DOCUMENTATION.md</code> / <code>EXPLANATION_GUIDE.md</code>	The formal report and this guide.

6. How a request actually flows (two end-to-end stories)

Story A — "Alice saves a document"

1. Alice types a document in the **Web App** and clicks *Create*.
2. The browser sends `POST /documents` with her **JWT** to the API.
3. The **Document Service** writes the document **and** version 1 into **PostgreSQL** in one transaction (so they can never disagree).
4. It writes a row into **CollaborationLog** (the audit trail).

5. It sends the document to **Elasticsearch** so it becomes searchable.
6. It clears any **Redis** search caches that are now out of date.
7. The API replies **201 Created** . (*This is the "write path", Figure 3 in the report.*)

Story B — "Bob searches for it"

1. Bob types "kubernetes" and the Web App calls **GET /search?q=kubernetes** .
2. The **Search Service** first checks **Redis** using a key that includes **Bob's user id** (so results are never shared between users with different permissions).
3. On a cache miss, it asks **PostgreSQL** which document IDs Bob is allowed to see.
4. It queries **Elasticsearch**, *filtered to only those IDs*, and gets back ranked results with highlights.
5. It stores the answer in **Redis** (with a time limit) and returns it.
6. If Bob searches the same thing again, step 2 returns instantly. (*This is the "search path", Figure 4 in the report.*)

The key insight in both stories: **permissions are enforced in the data path, not just hidden in the UI** — search literally cannot return a document Bob isn't allowed to see.

7. The six descriptive questions (Q1–Q6), answered simply

Q1 — Requirements Analysis

Functional (what it must do): register/login; create, edit, delete, version, tag documents; full-text search with ranking; share documents with specific people; comment and see activity; and **search must respect permissions**. **Non-functional (how well it must do it):** fast search (under ~200 ms), scale to millions of docs and many concurrent users, 99.9%+ availability, strong consistency for document content, secure (encrypted transport, hashed passwords, JWT), durable (backups), and maintainable. **Why the three highlighted ones matter:** - *Search efficiency* — if people can't find documents instantly, they stop using the system. An inverted index makes search scale. - *Collaboration* — versioning + comments keep knowledge trustworthy and current, not stale. - *Scalability* — an enterprise keeps growing; the design must scale without being rebuilt.

Q2 — System Architecture

A user → **Web App** → **stateless API** (Auth, Documents, Search, Collaboration, Indexing modules) → **data tier** (PostgreSQL = truth, Elasticsearch = search, Redis = cache, optional S3 = files). The API is stateless so we can run many copies behind a load balancer. See the simple diagram (Figure 1) and the detailed diagram (Figure 2) in the report.

Q3 — Document Management & Search Workflow

- **Create/Version:** each save writes the document plus a new immutable version row.
- **Index:** after saving, the document is pushed to Elasticsearch.
- **Search:** queries hit the cache first, then are permission-filtered and ranked by Elasticsearch.
- **Synchronize:** the indexing step and audit log keep search and history in step with the database.
- **Share securely:** sharing writes an ACL row; all reads and searches check it; everything travels over HTTPS with a JWT.

Q4 — Database Design

We use **PostgreSQL (SQL)** because the data is highly relational and needs ACID guarantees. Tables: **users** , **spaces** , **documents** , **document_versions** (append-only history), **permissions** (per-document ACL), **comments** ,

`collaboration_logs` (append-only audit). Full-text content is additionally stored in **Elasticsearch** (a NoSQL search store). This SQL-for-truth + NoSQL-for-search split is **polyglot persistence**. See the ER diagram (Figure 5) in the report.

Q5 — Algorithm & Implementation

Search is a **TF-IDF inverted index with cosine similarity** (`app/services/inverted_index.py`): 1. **Tokenize** text (lowercase, split, remove stop-words). 2. **Index**: build a map of `term → {document → how many times it appears}`. 3. **Weight**: rare words count more (IDF); frequent-in-a-doc words count more (TF). 4. **Rank**: score each document by cosine similarity to the query, so long documents aren't unfairly favoured. This is the same idea behind Lucene/Elasticsearch (which use the related BM25), just written out so you can read it.

Q6 — Scalability & Fault Tolerance

Scale: add more API replicas (stateless), shard/replicate Elasticsearch, use Postgres read replicas, cache aggressively with Redis, and move indexing to a queue so writes stay fast. **Faults**: - *Indexing delays* → Elasticsearch is near-real-time; for big loads, an async queue smooths spikes; the DB is always correct even if the index lags. - *Sync conflicts* → versioning keeps a full history; last-write creates a new version, nothing is lost. - *Storage outage* → replication + automated backups (Postgres PITR, ES snapshots); the search index can always be **rebuilt from PostgreSQL**.

8. System-design interview questions, answered *for this project*

CAP theorem — and which property would you prioritize?

CAP says that when a network **partition** (P) happens, a distributed system can guarantee **either Consistency (C) or Availability (A)**, not both. - A **CP** system stays consistent but may reject requests during a partition. - An **AP** system stays available but may serve slightly stale data.

Our choice is mixed, by data type: - For **document content, versions, and permissions** (the PostgreSQL side) we prioritize **CP — consistency**. You must never see a wrong version or bypass a permission, even if that means a brief unavailability during failover. - For **search results** (the Elasticsearch + Redis side) we accept **AP — availability with eventual consistency**. A brand-new document might take a second to appear in search; that's a fine trade for staying fast and always-on.

So: **CP where correctness matters (truth), AP where speed matters (search/cache)**.

Where would you use caching, and why?

We cache in **Redis** on the **read-heavy paths**: - **Search results**, keyed per user (search is the most frequent, most expensive operation). - **Hot document reads** (`doc:{id}`), so popular documents don't hit the database every time. Caching cuts latency from tens of milliseconds to microseconds and shields PostgreSQL/Elasticsearch from repeated identical work. We **invalidate** the cache on any write (edit, share) so users never see stale data. In a bigger build we'd also cache metadata lookups and (future) recommendations, and put a **CDN** in front of static assets.

How would you design the database sharding strategy?

Sharding = splitting one big database across many machines. Options and our pick: - **Hash-based** (shard by a hash of document id) — even spread, no hotspots; best default for documents. - **Range-based** (e.g. by date) — simple but creates hotspots on "today". - **Directory/tenant-based** (shard by organization/space) — **our preferred real-world choice for an enterprise knowledge base**, because each company's data stays together, which keeps their queries on one shard and makes per-tenant scaling and isolation easy. - **Geographic** (shard by region) — used when users are global, to keep data near them. Elasticsearch shards its index automatically by document id hash, which complements this.

How would you ensure fault tolerance?

- **Replication** of every store (Postgres streaming replicas, ES replica shards, Redis replicas/Sentinel).
- **Stateless API** so a dead replica is simply replaced.
- **Retries with idempotent indexing** (upserts keyed by id, so a retry is safe).
- **Failover** to a standby primary on database failure.
- **Circuit breakers / graceful degradation** — if search is down, the app still serves documents (and our code literally falls back to a local index).
- **Backups + monitoring + alerts** so problems are caught and recoverable.

How would you scale from 1 million to 100 million users?

- **Horizontal scaling**: many stateless API replicas behind a load balancer (auto-scaled).
- **Split modules into microservices** (Search and Indexing first — they have the heaviest, spikiest load).
- **Caching everywhere** + a **CDN** for static assets.
- **Asynchronous processing**: a message queue (Kafka/RabbitMQ) for indexing and notifications, so writes don't wait.
- **Distributed/sharded databases** + read replicas; Elasticsearch cluster with many shards.
- **Multi-region** deployment for global latency and disaster resilience.

What are the major bottlenecks in this architecture?

- **Database hotspots** (a popular shard or row) — mitigate with sharding, replicas, caching.
- **Network latency** between services — mitigate with co-location, batching, caching.
- **Cache misses / cold caches** — mitigate with cache warming and sensible TTLs.
- **Message-queue backlog** during write spikes — mitigate with more consumers and back-pressure.
- **Elasticsearch indexing pressure** under heavy writes — mitigate with async, bulk indexing.

How would you secure communication between services?

- **HTTPS/TLS** for everything in transit (client ↔ API and service ↔ service).
- **mTLS (mutual TLS)** between internal services so each side verifies the other.
- An **API Gateway** as the single guarded entry point (TLS, rate limiting, auth).
- **OAuth2 / JWT** tokens for user auth (we implement the JWT password flow); short-lived tokens limit damage if one leaks.
- **Secrets** kept out of code (env vars / a secrets manager), passwords hashed with **bcrypt**.

Explain eventual consistency with an example.

Eventual consistency means after a change, all copies of the data will agree *eventually*, but maybe not instantly. Classic example: a **social-media like count** — you like a post and see 101, but a friend on another server still sees 100 for a moment, until the update propagates. **In our project**: when Alice saves a document, PostgreSQL is updated immediately (strongly consistent), but **Elasticsearch becomes searchable about a second later** (its near-real-time refresh). For that brief window, search is *eventually* consistent with the database — an acceptable trade for fast, always-available search.

How would you monitor this system in production?

The classic **three pillars + alerts**: - **Metrics** (Prometheus/Grafana): request rate, error rate, latency (p50/p95/p99), cache hit ratio, queue depth, DB/ES health. - **Logs** (centralised, e.g. ELK): structured logs of requests, errors, and audit events (our `collaboration_logs` already records actions). - **Traces** (OpenTelemetry/Jaeger): follow a single request across services to find slow hops. - **Dashboards + alerts** tied to **SLOs/SLAs** (e.g. "alert if p95 search latency > 200 ms for 5 minutes"). We already expose a `/health` endpoint for liveness/readiness checks.

How would you design disaster recovery for this system?

- **Backups:** automated **Postgres PITR** (point-in-time recovery) and periodic **Elasticsearch snapshots** to object storage.
 - **Replication:** cross-region replicas of the database and search cluster.
 - **Multi-region deployment** with the ability to **fail over** to another region.
 - **Defined RPO/RTO:** *RPO* (how much data you can afford to lose) and *RTO* (how fast you must be back) drive backup frequency and failover automation — e.g. RPO of minutes via continuous replication, RTO of minutes via automated failover.
 - **Rebuildable derived data:** because Elasticsearch is derived from PostgreSQL, the entire search index can be **rebuilt** after a disaster — we never depend on it for the source of truth.
 - **Tested recovery:** run periodic **game-day / restore drills** so the plan actually works when needed.
-

9. Quick revision cheat-sheet

- **What:** an enterprise knowledge base (like Confluence/Notion).
- **Architecture:** modular monolith, microservices-ready; polyglot persistence; CQRS-lite (write to SQL, read search from ES).
- **Stores:** PostgreSQL = truth (CP), Elasticsearch = search (AP/eventual), Redis = cache.
- **Search algorithm:** TF-IDF inverted index + cosine similarity (same family as BM25).
- **Security:** HTTPS + JWT + bcrypt + RBAC + per-document ACLs; permissions enforced in the data path.
- **Scale:** stateless API replicas, ES sharding, Postgres read replicas, Redis caching, async indexing, CDN, multi-region.
- **Reliability:** replication, retries, idempotent indexing, backups (PITR + snapshots), graceful degradation, rebuildable index.
- **CAP pick:** CP for truth, AP for search/cache.
- **One-line pitch:** *"The API coordinates three stores — a database for truth, a search engine for speed, and a cache for repeated reads — behind a secure, stateless, horizontally scalable design."*